

Explication détaillée du code App.js

1. Imports et dépendances

javascript

```
import { useState, useEffect } from 'react';
```

```
import { recipes as initialRecipes } from './data/recipes';
```

- **React Hooks** : useState pour gérer l'état local, useEffect pour les effets de bord
- **Données initiales** : Import des recettes depuis un fichier de données séparé

2. Système de traductions multilingue

javascript

```
const translations = {
```

```
  fr: { /* français */ },
```

```
  en: { /* anglais */ },
```

```
  bassa: { /* langue locale camerounaise */ }
```

```
};
```

Fonctionnalités :

- Support de 3 langues : français, anglais, et bassa (langue camerounaise)
- Traductions complètes de l'interface utilisateur
- Facilite l'ajout de nouvelles langues

3. États de l'application (useState)

États principaux :

- recipes : Liste complète des recettes (initiales + utilisateur)
- searchTerm : Terme de recherche actuel
- selectedCuisine : Filtre de cuisine sélectionné
- favorites : IDs des recettes favorites
- currentLanguage : Langue actuelle de l'interface

États des modales :

- showShoppingList : Affichage de la liste de courses
- showRecipeScaler : Affichage du redimensionneur de recettes

- showUserRecipeForm : Affichage du formulaire d'ajout
- showDeliveryModal : Affichage de l'intégration livraison

4. Persistance des données (localStorage)

javascript

```
useEffect(() => {

  const savedFavorites = JSON.parse(localStorage.getItem('favorites') || '[]');

  const savedLanguage = localStorage.getItem('language') || 'fr';

  const savedUserRecipes = JSON.parse(localStorage.getItem('userRecipes') || '[]');

  // ...

}, []);
```

Données sauvegardées :

- **Favoris** : Persistance des recettes favorites
- **Langue** : Mémorisation de la préférence linguistique
- **Recettes utilisateur** : Sauvegarde des recettes ajoutées par l'utilisateur

5. Système de filtrage avancé

javascript

```
const filteredRecipes = recipes.filter(recipe => {

  const matchesSearch = recipe.name.toLowerCase().includes(searchTerm.toLowerCase()) ||

  recipe.ingredients.some(ingredient =>

    ingredient.toLowerCase().includes(searchTerm.toLowerCase())

  );

  const matchesCuisine = selectedCuisine === '' || recipe.cuisine === selectedCuisine;

  const matchesFavorites = !showFavoritesOnly || favorites.includes(recipe.id);

  return matchesSearch && matchesCuisine && matchesFavorites;

});
```

Critères de filtrage :

- **Recherche textuelle** : Nom de recette ET ingrédients

- **Filtre par cuisine** : Type de cuisine sélectionné
- **Filtre favoris** : Affichage des favoris uniquement

6. Fonctionnalités interactives

Gestion des favoris :

javascript

```
const toggleFavorite = (recipeId) => {
  setFavorites(prev =>
    prev.includes(recipeId)
      ? prev.filter(id => id !== recipeId)
      : [...prev, recipeId]
  );
};
```

Système de notation :

javascript

```
const rateRecipe = (recipeId, rating) => {
  // Calcul de la moyenne pondérée des notes
  const newAverageRating = ((recipe.averageRating * recipe.ratingsCount) + rating) /
    newRatingsCount;
};
```

7. Liste de courses intelligente

javascript

```
const addToShoppingList = (recipe) => {
  setShoppingListRecipes(prev => {
    if (!prev.find(r => r.id === recipe.id)) {
      return [...prev, recipe];
    }
    return prev;
  });
};
```

```
};
```

- **Évite les doublons** : Vérification avant ajout
- **Accumulation** : Agrégation des ingrédients de plusieurs recettes

8. Ajout de recettes utilisateur

javascript

```
const addUserRecipe = (newRecipe) => {  
  const recipeWithId = {  
    ...newRecipe,  
    id: Date.now(), // ID unique basé sur timestamp  
    averageRating: 0,  
    ratingsCount: 0,  
    author: 'Vous',  
    isUserRecipe: true // Marqueur pour distinguer les recettes utilisateur  
  };  
  
  setUserRecipes(prev => [...prev, recipeWithId]);  
  setRecipes(prev => [...prev, recipeWithId]);  
};
```

9. Optimisations UX

Titre dynamique :

javascript

```
useEffect(() => {  
  document.title = `${t.title} (${filteredRecipes.length} recettes)`;  
}, [filteredRecipes.length, t.title]);
```

- Met à jour le titre de l'onglet avec le nombre de recettes affichées

10. Architecture des composants

Composants modulaires :

- RecipeCard : Affichage individuel des recettes

- FilterBar : Barre de recherche et filtres
- ShoppingList : Gestion de la liste de courses
- RecipeScaler : Ajustement des portions
- LanguageSelector : Sélection de langue
- UserRecipeForm : Formulaire d'ajout de recettes
- DeliveryIntegration : Intégration avec services de livraison

11. Fonctionnalités avancées

Mise à l'échelle des recettes :

- Ajustement automatique des quantités selon le nombre de portions
- Calcul proportionnel des ingrédients

Intégration livraison :

- Connexion avec des services de livraison locaux
- Export automatique de la liste de courses

Support multilingue complet :

- Interface entièrement traduite
- Support des langues locales camerounaises

12. Gestion d'état complexe

L'application utilise un pattern de gestion d'état sophistiqué :

- **État local** pour l'interactivité immédiate
- **Persistence** pour les données importantes
- **Synchronisation** entre composants via props et callbacks

Cette architecture permet une expérience utilisateur fluide tout en maintenant la performance et la maintenabilité du code.

Explication détaillée du composant LanguageSelector

1. Import de React

```
import React from 'react';
```

Rôle : Cette ligne importe la bibliothèque React nécessaire pour créer des composants React.

- `import` : Mot-clé ES6 pour importer des modules
- `React` : L'objet principal de la bibliothèque React
- `from 'react'` : Spécifie le package d'où importer

2. Déclaration du composant

```
const LanguageSelector = ({ currentLanguage, onLanguageChange }) => {
```

Structure :

- `const` : Déclare une constante (le composant ne sera pas réassigné)
- `LanguageSelector` : Nom du composant (convention : PascalCase)
- `({ currentLanguage, onLanguageChange })` : **Destructuration des props**
 - `currentLanguage` : Prop qui contient la langue actuellement sélectionnée
 - `onLanguageChange` : Prop qui est une fonction callback pour gérer le changement de langue
- `=> {` : Syntaxe de fonction fléchée ES6

3. Définition des données des langues

```
const languages = [  
  
  { code: 'fr', name: 'Français', flag: 'FR' },  
  
  { code: 'en', name: 'English', flag: 'GB' },  
  
  { code: 'bassa', name: 'Bassa', flag: 'CM' }  
];
```

Structure :

- `const languages` : Tableau constant contenant les langues disponibles
- Chaque objet du tableau contient :
 - `code` : Identifiant unique de la langue (utilisé comme valeur)
 - `name` : Nom affiché de la langue
 - `flag` : Emoji du drapeau pour l'affichage visuel

4. Rendu du composant (return)

```
return (  
  <div className="language-selector">
```

Explication :

- return : Retourne le JSX à rendre
- <div className="language-selector"> : Conteneur principal avec classe CSS

5. Élément select

```
<select  
  value={currentLanguage}  
  onChange={(e) => onLanguageChange(e.target.value)}  
  className="language-select"  
>
```

Attributs détaillés :

- value={currentLanguage} :
 - **Composant contrôlé** : La valeur est liée à l'état du parent
 - Affiche la langue actuellement sélectionnée
- onChange={(e) => onLanguageChange(e.target.value)} :
 - **Gestionnaire d'événement** déclenché lors du changement de sélection
 - e : Objet event contenant les informations sur l'événement
 - e.target.value : La nouvelle valeur sélectionnée
 - onLanguageChange() : Appelle la fonction callback du parent avec la nouvelle valeur
- className="language-select" : Classe CSS pour le styling

6. Génération dynamique des options

```
{languages.map(lang => (  
  <option key={lang.code} value={lang.code}>  
    {lang.flag} {lang.name}  
  </option>  
))}
```

Décomposition :

- `{languages.map(...)}` :
 - Utilise la méthode `map()` pour transformer chaque élément du tableau
 - Les accolades `{}` permettent d'insérer du JavaScript dans le JSX
- `lang =>` : Paramètre de la fonction `map` (chaque objet langue)
- `<option key={lang.code} value={lang.code}>` :
 - `key={lang.code}` : **Clé unique** requise par React pour optimiser le rendu
 - `value={lang.code}` : Valeur de l'option (ce qui sera envoyé lors de la sélection)
- `{lang.flag} {lang.name}` : Contenu affiché (drapeau + nom de la langue)

7. Export du composant

`export default LanguageSelector;`

Rôle :

- `export default` : Exporte le composant comme export par défaut du module
- Permet à d'autres fichiers d'importer ce composant avec `import LanguageSelector from './LanguageSelector'`

Concepts React utilisés

Props (Propriétés)

- **currentLanguage** : Données reçues du composant parent
- **onLanguageChange** : Fonction callback pour communiquer avec le parent

Composant contrôlé

- La valeur du `select` est contrôlée par les props, pas par l'état interne
- Suit le pattern "single source of truth" de React

Rendu de listes

- Utilisation de `map()` pour générer dynamiquement les options
- Importance de la prop `key` pour les performances

Flux de données

1. **Parent → Enfant** : `currentLanguage` passe la langue active
2. **Enfant → Parent** : `onLanguageChange` remonte la nouvelle sélection

3. **Re-rendu** : Le parent met à jour son état, ce qui re-rend le composant

Utilisation typique

// Dans le composant parent

```
const [language, setLanguage] = useState('fr');
```

```
<LanguageSelector
```

```
  currentLanguage={language}
```

```
  onLanguageChange={setLanguage}
```

```
/>
```

Ce composant est un exemple parfait de composant React réutilisable et bien structuré !

Analyse complète de l'application de recettes culinaires

Vue d'ensemble du projet

Cette application React est une plateforme de recettes culinaires camerounaises avec les fonctionnalités suivantes :

- Interface multilingue (Français, Anglais, Bassa)
- Gestion de recettes avec notation
- Liste de courses automatique
- Ajustement des portions
- Intégration livraison
- Formulaire d'ajout de recettes personnalisées

1. LanguageSelector - Sélecteur de langue

```
const LanguageSelector = ({ currentLanguage, onLanguageChange }) => {
```

```
  const languages = [
```

```
    { code: 'fr', name: 'Français', flag: 'FR' },
```

```
    { code: 'en', name: 'English', flag: 'GB' },
```

```
{ code: 'bassa', name: 'Bassa', flag: 'CM' }  
];
```

Fonction : Permet de changer la langue de l'interface

- **Props reçues :** currentLanguage (langue actuelle), onLanguageChange (fonction de changement)
- **Langues supportées :** Français, Anglais, Bassa (langue locale camerounaise)
- **Interface :** Menu déroulant avec drapeaux et noms des langues

2. DeliveryIntegration - Service de livraison

Structure générale

```
const DeliveryIntegration = ({ isOpen, onClose, shoppingList, language }) => {  
  const [selectedService, setSelectedService] = useState("");  
  const [deliveryInfo, setDeliveryInfo] = useState({  
    address: "", phone: "", notes: ""  
  });
```

Fonction principale : Interface pour commander des ingrédients via des services de livraison

Traductions multilingues

Le composant contient des objets de traduction pour 3 langues :

```
const translations = {  
  fr: { title: 'Commander vos Courses', ... },  
  en: { title: 'Order Your Groceries', ... },  
  bassa: { title: 'Sòmba Bidyô Byu', ... }  
};
```

Services de livraison disponibles

```
const deliveryServices = [  
  {  
    id: 'glovo',  
    name: 'Glovo',  
    deliveryTime: '30-45 mins',
```

```

    minOrder: '2000 FCFA',
    deliveryFee: '500 FCFA'
  },
  // Jumia Food, Marché Local...
];

```

Fonctionnalités clés

- **Sélection de service** : L'utilisateur choisit parmi Glovo, Jumia Food, ou Marché Local
- **Formulaire de livraison** : Adresse, téléphone, instructions spéciales
- **Calcul automatique** : Coût estimé basé sur le nombre d'ingrédients
- **Résumé de commande** : Affichage des ingrédients par recette

3. UserRecipeForm - Formulaire d'ajout de recettes

État du formulaire

```

const [formData, setFormData] = useState({
  name: "", cuisine: 'Camerounaise', difficulty: 'Facile',
  cookTime: "", servings: 1, ingredients: [],
  instructions: "", tags: "", histoire: "", image: ""
});

```

Gestion dynamique des ingrédients

```

const addIngredient = () => {
  setFormData(prev => ({
    ...prev,
    ingredients: [...prev.ingredients, ""]
  }));
};

const removeIngredient = (index) => {
  if (formData.ingredients.length > 1) {
    const newIngredients = formData.ingredients.filter((_, i) => i !== index);

```

```

    setFormData(prev => ({ ...prev, ingredients: newIngredients }));
  }
};

```

Fonctionnalités :

- **Champs dynamiques** : Ajout/suppression d'ingrédients
- **Validation** : Vérification des champs obligatoires
- **Catégorisation** : Sélection du type de cuisine et difficulté
- **Histoire culturelle** : Champ pour l'origine/histoire de la recette

4. RecipeScaler - Ajustement des portions

Calcul du facteur d'échelle

```

const scaleFactor = useMemo(() => {
  if (!recipe) return 1;
  return newServings / recipe.servings;
}, [newServings, recipe]);

```

Mise à l'échelle intelligente des ingrédients

```

const scaleIngredient = useCallback((ingredient) => {
  const patterns = [
    /^(\d+(?:\.\d+)?)\s*(kg|g|l|ml|cl|cuillères?|tasses?|pincées?)\s*(.+)$/i,
    /^(\d+(?:\.\d+)?)\s*(.+)$/i,
    /^(\d+(?:\d+)?)\s*(.+)$/i
  ];

  for (const pattern of patterns) {
    const match = ingredient.match(pattern);
    if (match) {
      const quantity = parseFloat(match[1]);
      if (!isNaN(quantity)) {
        const scaledQuantity = (quantity * scaleFactor).toFixed(quantity < 1 ? 2 : 1);

```

```

    // Reconstruction de l'ingrédient avec nouvelle quantité
  }
}
}, [scaleFactor]);

```

Caractéristiques avancées :

- **Parsing intelligent** : Reconnaissance de différents formats de quantités
- **Unités de mesure** : Support des unités métriques et traditionnelles
- **Boutons rapides** : Facteurs prédéfinis (×0.5, ×1, ×1.5, etc.)
- **Nutrition ajustée** : Recalcul automatique des valeurs nutritionnelles

5. ShoppingList - Liste de courses intelligente

Consolidation des ingrédients

```

const shoppingList = useMemo(() => {
  const ingredientCount = {};

  recipes.forEach(recipe => {
    recipe.ingredients.forEach(ingredient => {
      const cleanIngredient = ingredient.toLowerCase().trim();
      if (ingredientCount[cleanIngredient]) {
        ingredientCount[cleanIngredient].count += 1;
        ingredientCount[cleanIngredient].recipes.push(recipe.name);
      } else {
        ingredientCount[cleanIngredient] = {
          original: ingredient, count: 1, recipes: [recipe.name]
        };
      }
    });
  });
});

```

```
return ingredientCount;
}, [recipes]);
```

Catégorisation automatique

```
const ingredientCategories = {
  'Légumes': ['gombo', 'épinard', 'oseille', 'légumes', 'oignon', 'piment'],
  'Viandes & Poissons': ['viande', 'bœuf', 'porc', 'poisson', 'crevettes'],
  'Céréales & Légumineuses': ['semoule', 'couscous', 'haricots', 'arachides'],
  'Tubercules': ['taro', 'banane', 'plantain'],
  'Condiments': ['huile de palme', 'sel', 'poivre', 'maggi', 'pâte'],
  'Autres': []
};
```

Fonctionnalités :

- **Déduplication** : Évite les doublons d'ingrédients
- **Comptage** : Indique combien de recettes utilisent chaque ingrédient
- **Deux vues** : Par recette ou par catégorie d'aliments
- **Export/Impression** : Sauvegarde en fichier texte ou impression

6. RecipeCard - Carte de recette complète

Système de notation

```
const handleRating = (rating) => {
  if (!hasRated) {
    setUserRating(rating);
    setHasRated(true);
    onRateRecipe(recipe.id, rating);
  }
};
```

```
const renderStars = (rating, interactive = false) => {
```

```

return Array.from({ length: 5 }, (_, i) => (
  <span
    key={i}
    className={`star ${i < rating ? 'filled' : ''} ${interactive ? 'interactive' : ''}`}
    onClick={interactive ? () => handleRating(i + 1) : undefined}
  >
    ★
  </span>
));
};

```

Affichage des informations

- **Métadonnées** : Difficulté, temps de cuisson, portions
- **Auteur et région** : Informations sur l'origine
- **Tags** : Catégorisation de la recette
- **Instructions extensibles** : Texte tronqué avec option "Voir plus"
- **Valeurs nutritionnelles** : Calories, protéines, glucides, etc.

Architecture technique

Gestion d'état

- **useState** pour l'état local des composants
- **useMemo** pour les calculs coûteux (mise à l'échelle, catégorisation)
- **useCallback** pour optimiser les fonctions réutilisées

Internationalisation

- Objets de traduction centralisés dans chaque composant
- Support de 3 langues incluant une langue locale (Bassa)
- Interface adaptée au contexte camerounais

UX/UI avancée

- **Modales** : Interface non-intrusive pour les fonctionnalités
- **Validation** : Vérification des formulaires

- **Feedback utilisateur** : Messages de confirmation/erreur
- **Responsive** : Adaptation mobile (via classes CSS)

Intégration e-commerce

- **Services de livraison réels** : Glovo, Jumia Food
- **Prix en FCFA** : Monnaie locale camerounaise
- **Adresses locales** : Adaptées à Yaoundé

Points forts du code

1. **Modularité** : Chaque composant a une responsabilité claire
2. **Réutilisabilité** : Props configurables pour différents contextes
3. **Performance** : Optimisations avec useMemo/useCallback
4. **Accessibilité** : Labels et structure sémantique
5. **Internationalisation** : Support multilingue natif
6. **Culture locale** : Adaptation aux habitudes camerounaises

Cette application représente un excellent exemple d'application React moderne avec une approche centrée sur l'utilisateur local et les besoins culturels spécifiques.